

《动手学深度学习》

PyTorch 版学习笔记

内容提炼试作

2026 年 9 月 1 日

试作说明

本 PDF 用于检验笔记的内容密度、排版和编译流程，仅提炼原书第二章“预备知识”。章内目录严格沿用 PyTorch 版原始顺序：数据操作、数据预处理、线性代数、微积分、自动微分、概率、查阅文档。正文只保留理解后续深度学习所需的概念、公式、PyTorch 操作和易混淆点。

目录

试作说明	i
第二章 预备知识	1
2.1 数据操作	1
2.2 数据预处理	2
2.3 线性代数	3
2.4 微积分	4
2.5 自动微分	5
2.6 概率	6
2.7 查阅文档	7

第二章 预备知识

核心摘要

本章建立后续深度学习所需的最小工具链：用张量表示和操作数据，用 pandas 清洗表格，用线性代数描述模型，用微积分与自动微分训练参数，用概率描述不确定性，并能主动查阅 PyTorch API。

2.1 数据操作

深度学习中的数据主要表示为张量（tensor）。标量是零轴张量，向量是一轴张量，矩阵是二轴张量，更高维对象仍统一称为张量。PyTorch 的包名是 `torch`。

形状与创建

操作	含义
<code>torch.arange(n)</code>	创建从 0 到 $n - 1$ 的一维张量
<code>x.shape</code>	返回每个轴的长度
<code>x.numel()</code>	返回元素总数，即各轴长度之积
<code>x.reshape(3, 4)</code>	改变观察数据的形状，不改变元素及其数量
<code>zeros</code> / <code>ones</code> / <code>randn</code>	创建全零、全一或标准正态随机张量

`reshape` 中可用 `-1` 让框架自动推断一个轴的长度，但新旧形状的元素总数必须相同。

运算、广播和索引

同形状张量的 `+` `-` `*` `/` `**` 默认逐元素运算；其中 `*` 不是矩阵乘法。形状不同时，PyTorch 可沿长度为 1 的轴广播。例如 3×1 与 1×2 相加后得到 3×2 。

```
import torch
x = torch.arange(12).reshape(3, 4)
a = torch.arange(3).reshape(3, 1)
b = torch.arange(2).reshape(1, 2)
c = a + b

last_row = x[-1]
block = x[1:3]          # includes 1, excludes 3
x[0:2, :] = 12
```

内存与互操作

$Y = Y + X$ 会创建新对象； $Y += X$ 或 $Y[:]=Y+X$ 尽量复用原内存。张量与 NumPy 数组可共享底层内存，因此一侧的原地修改可能影响另一侧。单元素张量可用 `item()` 转为 Python 标量。

易混淆点

- `shape` 描述各轴长度，`numel()` 描述元素总数。
- 广播是在运算时扩展形状；`reshape` 是重新解释同一批元素的形状。
- `*` 是逐元素乘法，矩阵乘法使用 `torch.mm` 或 `@`。

2.2 数据预处理

真实数据通常先以 CSV 等表格形式出现。基本流程是：

原始数据 → pandas 清洗 → 数值矩阵 → PyTorch 张量。

1. 用 `pd.read_csv` 读取数据，行表示样本，列表示属性。
2. 将输入列与目标列分开。
3. 数值列的缺失值可用该列均值填充。
4. 类别列可做独热编码，并将缺失本身视为一个类别。
5. 所有字段数值化后，再转换为张量。

```
import pandas as pd
import torch

data = pd.read_csv(data_file)
```

```
inputs, outputs = data.iloc[:, 0:2], data.iloc[:, 2]
inputs = inputs.fillna(inputs.mean(numeric_only=True))
inputs = pd.get_dummies(inputs, dummy_na=True)

X = torch.tensor(inputs.to_numpy(dtype=float))
y = torch.tensor(outputs.to_numpy(dtype=float))
```

易混淆点

数值缺失和类别缺失的含义不同：前者可用统计量插补，后者通常需要单独编码。转换为张量前应确认所有列均已数值化且数据类型一致。

2.3 线性代数

对象与形状

- 标量 $x \in \mathbb{R}$ ：单个数。
- 向量 $\mathbf{x} \in \mathbb{R}^n$ ： n 个标量组成的一列。
- 矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ ： m 行、 n 列。
- 张量：向量、矩阵及更高阶数组的统一称呼。

矩阵转置交换行列，记作 $\mathbf{A}^\top$ ；若 $\mathbf{A} = \mathbf{A}^\top$ ，则 $\mathbf{A}$ 为对称矩阵。

逐元素运算与降维

Hadamard 积是逐元素乘法：

$$\mathbf{A} \odot \mathbf{B} = [a_{ij}b_{ij}]_{m \times n}.$$

求和或平均会沿指定轴降维；`keepdim=True` 可保留长度为 1 的轴，便于后续广播。累积和 `cumsum` 不减少轴数。

点积与矩阵乘法

$$\begin{aligned}\mathbf{x}^\top \mathbf{y} &= \sum_{i=1}^d x_i y_i, \\ (\mathbf{A}\mathbf{x})_i &= \mathbf{a}_i^\top \mathbf{x}, \\ c_{ij} &= \mathbf{a}_i^\top \mathbf{b}_j, \quad \mathbf{C} = \mathbf{A}\mathbf{B}.\end{aligned}$$

若 $\mathbf{A} \in \mathbb{R}^{n \times k}$ 、 $\mathbf{B} \in \mathbb{R}^{k \times m}$ ，则 $\mathbf{AB} \in \mathbb{R}^{n \times m}$ 。矩阵乘法要求中间维度 k 相等。

范数

$$\begin{aligned}\|\mathbf{x}\|_1 &= \sum_i |x_i|, \\ \|\mathbf{x}\|_2 &= \sqrt{\sum_i x_i^2}, \\ \|\mathbf{X}\|_F &= \sqrt{\sum_i \sum_j x_{ij}^2}.\end{aligned}$$

范数把向量或矩阵映射为表示“大小”的标量。训练目标常用范数描述预测与真值的距离。

易混淆点

- 向量的“维度”常指分量个数；张量的“维度”也可能指轴数，应结合语境判断。
- 表格中通常一行是一个样本，而数学向量默认写成列向量。
- $\mathbf{A} * \mathbf{B}$ 是 Hadamard 积； $\mathbf{A} @ \mathbf{B}$ 才是矩阵乘法。

2.4 微积分

训练模型通常意味着寻找使损失函数最小的参数。优化衡量对训练数据的拟合过程；泛化衡量模型在未见数据上的表现，二者不能等同。

导数、偏导和梯度

单变量函数的导数为

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

多元标量函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 对每个输入求偏导，组成梯度向量：

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1} \quad \dots \quad \frac{\partial f}{\partial x_n} \right]^\top.$$

两个后续常用结果是

$$\nabla_{\mathbf{x}} \|\mathbf{x}\|_2^2 = 2\mathbf{x}, \quad \nabla_{\mathbf{X}} \|\mathbf{X}\|_F^2 = 2\mathbf{X}.$$

链式法则

若 $y = f(u)$ 且 $u = g(x)$, 则

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}.$$

若多个中间变量共同影响输出, 各计算路径的贡献需要相加:

$$\frac{\partial y}{\partial x_i} = \sum_{j=1}^m \frac{\partial y}{\partial u_j} \frac{\partial u_j}{\partial x_i}.$$

深层网络本质上是多层复合函数, 反向传播正是链式法则的系统化应用。

2.5 自动微分

PyTorch 在前向计算时记录计算图, 在反向阶段沿图应用链式法则, 把梯度写入叶子张量的 `grad` 属性。

```
x = torch.arange(4.0)
x.requires_grad_(True)
y = 2 * torch.dot(x, x)
y.backward()
assert torch.allclose(x.grad, 4 * x)
```

标准流程是:

启用梯度追踪 → 前向计算 → backward → 读取梯度.

四个关键规则

1. 梯度会累积: 再次反向传播前通常调用 `x.grad.zero_()`。
2. 非标量需先聚合: 例如 `y.sum().backward()`; 这等价于对各输出分量的梯度求和。
3. 分离计算: `detach()` 保留数值但切断该路径的梯度传播。
4. 控制流可求导: PyTorch 对实际执行的 `if/while` 路径中的可微运算求导, 不对离散分支选择本身求导。

易混淆点

`detach()` 不改变张量数值, 只改变梯度流向。默认情况下, 反向传播后计算图会释放; 重复使用同一张图需明确保留, 但普通训练循环通常每轮重新构建计算图。

2.6 概率

机器学习需要在信息不完整时做预测，概率提供描述不确定性的统一语言。

样本空间、事件与随机变量

样本空间 $\mathcal{S}$ 包含所有可能结果，事件 $\mathcal{A}$ 是 $\mathcal{S}$ 的子集。概率满足

$$\begin{aligned} P(\mathcal{A}) &\geq 0, \\ P(\mathcal{S}) &= 1, \\ P\left(\bigcup_i \mathcal{A}_i\right) &= \sum_i P(\mathcal{A}_i) \quad (\mathcal{A}_i \text{ 两两互斥}). \end{aligned}$$

随机变量 X 把随机结果映射为数值。离散变量可直接讨论 $P(X = x)$ ；连续变量在单点处概率通常为零，需要通过密度和区间概率描述。

联合、条件与贝叶斯公式

$$\begin{aligned} P(A, B) &= P(B | A)P(A), \\ P(B | A) &= \frac{P(A, B)}{P(A)}, \\ P(A | B) &= \frac{P(B | A)P(A)}{P(B)}, \\ P(B) &= \sum_A P(A, B). \end{aligned}$$

最后一式称为边际化：对未关注变量的所有可能取值求和。若 A 与 B 独立，则

$$A \perp B \iff P(A, B) = P(A)P(B).$$

条件独立则要求在给定 C 后可分解：

$$A \perp B | C \iff P(A, B | C) = P(A | C)P(B | C).$$

期望与方差

$$\begin{aligned} \mathbb{E}[X] &= \sum_x xP(X = x), \\ \text{Var}(X) &= \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2. \end{aligned}$$

期望描述分布中心，方差描述围绕期望的波动程度；标准差是方差的平方根，与原变量具有相同量纲。

易混淆点

- 独立不等于互斥：独立指概率可分解，互斥指两个事件不能同时发生。
- 联合概率只有在独立条件下才能直接写成边际概率之积。
- 低先验概率事件即使检测呈阳性，后验概率也未必很高。

2.7 查阅文档

学习框架的重点不是记住所有 API，而是能发现和核验所需接口。

工具	用途
<code>dir(module)</code>	列出模块中的函数、类和属性
<code>help(torch.ones)</code>	查看函数签名、参数和文档字符串
<code>torch.ones?</code>	在 Jupyter 中查看帮助
<code>torch.ones??</code>	在可用时查看 Python 实现源码

双下划线包围的名称通常是 Python 特殊对象，单下划线开头的名称通常是内部实现。遇到版本差异时，应以当前版本的 PyTorch 官方文档为准。

本章最小闭环

读入并清洗数据，将其变成张量；用线性代数完成前向计算；用损失函数度量误差；由自动微分得到梯度并更新参数；用概率解释预测的不确定性；遇到未知 API 时主动查阅官方文档。